



Herencia: especializar sin copiar

El problema

Un sistema de cobros para un local. Cobra con débito y con crédito, y cada medio de pago es una clase: `PagoDebito` y `PagoCredito`. Cada pago referencia la `Venta` que está pagando (una clase mínima: descripción y total) y recibe su número de operación del sistema. `PagoCredito` nació como copia de `PagoDebito`: mismos atributos, mismas validaciones, mismo comprobante, más las cuotas y su recargo.

Este es `PagoDebito`, recortado:

```
public class PagoDebito {

    private static final double ARANCEL = 0.008;    // 0.8% que retiene la tarjeta

    private final int numeroOperacion;
    private final Venta venta;
    private final String banco;
    private final String ultimos4;

    public PagoDebito(int numeroOperacion, Venta venta, String banco, String
ultimos4) {
        if (venta == null) {
            throw new IllegalArgumentException("La venta es obligatoria");
        }
        if (banco == null || banco.isBlank()) {
            throw new IllegalArgumentException("El banco es obligatorio");
        }
        // ... validacion de ultimos4 ...
        this.numeroOperacion = numeroOperacion;
        this.venta = venta;
        this.banco = banco;
        this.ultimos4 = ultimos4;
    }

    public double montoAcreditado() {
        return this.venta.getTotal() * (1 - ARANCEL);
    }

    public void mostrarComprobante() {
        System.out.println("Operacion nro " + this.numeroOperacion);
        System.out.println("Detalle: " + this.venta.getDescripcion());
        System.out.println("Monto: $" + this.venta.getTotal());
        System.out.println("Debito " + this.banco + " terminada en " +
```



```
this.ultimos4);  
    }  
}
```

PagoCredito es ese mismo código, más las cuotas: el atributo, su validación, el recargo y una línea más en el comprobante. Y un arancel distinto. No hay ningún bug. Todavía.

Ahora el local va a aceptar pagos con QR. ¿Qué hacemos? ¿Copiamos PagoDebito por segunda vez y le cambiamos banco por billetera?

Pensemos qué significa esa copia. La regla "la venta es obligatoria" pasa a existir tres veces. El día que el comprobante cambie de formato, hay que acordarse de tocar tres clases, y el compilador no avisa si tocamos dos: las tres versiones compilan, cada una por su lado. Las reglas que hoy son una sola van divergiendo en silencio, hasta que un cliente muestra dos tickets con formato distinto del mismo local.

El costo no es escribir la copia: son dos minutos. El costo es que desde la copia en adelante cada cambio vale triple y cada olvido es invisible.

Lo que hace falta es que lo común a los medios de pago se escriba una vez y cada medio de pago agregue solo lo suyo. Eso es la herencia, **reutilización de código**.

Una clase que es otra clase

Los medios de pago comparten lo mismo, un número de operación, una venta, un comprobante que mostrar y los dos son, literalmente, un pago. El QR que viene, también.

Esa frase es la clave. Un pago con QR **es un** pago. Cuando eso es verdad, la parte común se escribe una sola vez en una clase Pago y cada medio de pago la hereda.

El vocabulario:

- Pago es la **superclase** (también: clase padre, clase base). Modela lo general.
- PagoDebito, PagoCredito y PagoQR, y los que vengan, son **subclases** (clases hijas, clases derivadas). Modelan lo especializado.

La subclase no copia el código del padre: lo tiene. Un objeto PagoQR es un objeto Pago con algo más.

No existe la herencia solo por tener los mismos atributos o tener los mismos métodos. Entre las clases tiene que existir la relación "**es un**". Si suena forzada, no es herencia. No es solo sintaxis, necesita de la semántica (lo que significa).

La sintaxis: extends

Lo genérico, escrito una sola vez:

```
public class Pago {
```



```
private final int numeroOperacion;
private final Venta venta;

public Pago(int numeroOperacion, Venta venta) {
    if (venta == null) {
        throw new IllegalArgumentException("La venta es obligatoria");
    }
    this.numeroOperacion = numeroOperacion;
    this.venta = venta;
}

public double getMonto() {
    return this.venta.getTotal();
}

public void mostrarComprobante() {
    System.out.println("Operacion nro " + this.numeroOperacion);
    System.out.println("Detalle: " + this.venta.getDescripcion());
    System.out.println("Monto: $" + this.getMonto());
}
}
```

Y este es el archivo completo del medio de pago nuevo:

```
public class PagoQR extends Pago {

    private final String billetera; // alias extraido del codigo QR

    public PagoQR(int numeroOperacion, Venta venta, String codigoQR) {
        super(numeroOperacion, venta); // primero se construye el Pago
        this.billetera = extraerBilletera(codigoQR);
    }

    // El codigo escaneado trae varios datos separados por ";"
    // por ejemplo: "PAGO;lapampa.kiosco;ARS"
    private static String extraerBilletera(String codigoQR) {
        if (codigoQR == null || codigoQR.split(";").length != 3) {
            throw new IllegalArgumentException("El codigo QR no es valido");
        }
        return codigoQR.split(";")[1];
    }

    public String armarSolicitudCobro() {
        // el mensaje que se le manda a la API de la billetera (simulado)
        return "{billetera: " + this.billetera + ", monto: " + this.getMonto() +
            "}";
    }
}
```



```
}

@Override
public void mostrarComprobante() {
    super.mostrarComprobante(); // el encabezado comun
    System.out.println("Billetera: " + this.billetera);
}
}
```

Con `extends`, `PagoQR` hereda todos los atributos y métodos de `Pago`, y la regla "la venta es obligatoria" sigue escrita en un solo lugar. Si comparamos con `PagoDebito`: no se repiten las validaciones, ni el número de operación, ni el encabezado del comprobante. Escribe exactamente lo que un QR tiene de propio: decodificar el código escaneado, armar la solicitud de cobro para la API de la billetera virtual y su línea del comprobante.

Los miembros `private` del padre no se heredan, pero tampoco desaparecen. Adentro de cada objeto `PagoQR` está la parte `Pago` completa, con su venta. Lo que no puede hacer el código de `PagoQR` es llamarla, porque `private` significa "solo el código de la clase `Pago`". Para usarla tiene el método `getMonto()`, que sí heredó.

Para construir un hijo hay que construir primero al padre

Los constructores no se heredan, cada clase escribe el suyo pero adentro de un `PagoQR` está la parte `Pago`, y hay que inicializarla. Para eso está `super(...)`: la llamada al constructor del padre.

```
public PagoQR(int numeroOperacion, Venta venta, String codigoQR) {
    super(numeroOperacion, venta); // primero se construye el Pago
    this.billetera = extraerBilletera(codigoQR);
}
```

Tres reglas para memorizar:

- `super(...)` tiene que ser la primera instrucción del constructor.
- Si el padre tiene un constructor sin argumentos, Java lo llama solo y no hace falta escribir nada.
- Si el padre no tiene constructor sin argumentos, escribir `super(...)` es obligatorio. `Pago` valida la venta, así que no tiene constructor vacío: sin `super(...)`, no compila.

Ese paso obligatorio es el que garantiza que las validaciones comunes corran siempre: ningún pago, del medio que sea, puede instanciarse sin pasar por el constructor de `Pago`.

El orden de ejecución es siempre el mismo, primero corre el constructor del padre, después el del hijo.



protected

PagoQR necesita el monto, la solicitud de cobro que le manda a la API de la billetera lo lleva adentro. Pero `venta` es `private` de `Pago`. La tentación es cambiarla a `protected`.

`protected` es un nivel intermedio de visibilidad: accesible desde la propia clase, desde las clases del mismo paquete y, sobre todo, **desde cualquier subclase**.

Antes de tocarlo, la pregunta: ¿hace falta? Con el `getMonto()` que ya heredó, alcanza.

```
public String armarSolicitudCobro() {  
    // el mensaje que se le manda a la API de la billetera (simulado)  
    return "{billetera: " + this.billetera + ", monto: " + this.getMonto() + "}";  
}
```

Cuando un atributo pasa a `protected`, el padre deja de ser el único responsable de su estado. Cualquier subclase, las de hoy y las que alguien escriba dentro de tres años, puede modificarlo sin pasar por sus reglas. Es exactamente lo que evitamos en la clase 3.

La regla práctica es la misma que con `static`: **los atributos siguen siendo `private`**. `protected` sirve para métodos que el padre quiere ofrecerles a sus hijos y a nadie más. Un atributo `protected` es la excepción que hay que poder justificar.

Especializar sobrescribiendo métodos

Cada medio de pago imprime su comprobante distinto, el QR muestra la billetera; el débito, el banco y los últimos dígitos de la tarjeta. Pero el encabezado (número de operación, detalle, monto) es igual para todos.

Sobrescribir es volver a escribir en la subclase un método que ya existe en la superclase, con la misma firma. Y `super.metodo()` permite ejecutar la versión del padre desde adentro de la del hijo.

```
@Override  
public void mostrarComprobante() {  
    super.mostrarComprobante(); // el encabezado comun  
    System.out.println("Billetera: " + this.billetera);  
}
```

No se pierde nada del padre, se reutiliza y se le agrega.

La anotación `@Override` no es obligatoria, pero es útil, le pide al compilador que verifique que ese método realmente existe en el padre.

Cuándo NO hay que heredar

La herencia es tentadora porque reutiliza código. Pero no hay que usarla solo porque conviene, es necesario tener la relación “es un”.



Por ejemplo, `Pago` necesita los datos de la venta: la descripción, el total. Se podría escribir `class Pago extends Venta`, compila y deja todos esos datos disponibles sin escribir una línea. El problema es que un pago no es una venta.

Un pago **tiene** una venta. Y eso es exactamente lo que venimos haciendo, guardar una referencia a otro objeto como atributo, `private final Venta venta`. Se llama **composición**, y es la respuesta correcta cada vez que la relación es **"tiene un"**.

	Herencia ("es un")	Composición ("tiene un")
Cuándo	La subclase es un caso particular de la superclase.	Un objeto necesita a otro para trabajar.
Ejemplo	Un pago con QR es un pago.	Un pago tiene una venta.
En Java	<code>class PagoQR extends Pago</code>	<code>private Venta venta;</code>
Qué gana	Reutiliza y además se trabaja como un <code>Pago</code> .	Reutiliza sin atarse: la venta puede cambiar.

Herencia simple y el problema del diamante

En Java una clase hereda de una sola superclase. Otros lenguajes, como C++ o Python, permiten heredar de varias. Java lo prohíbe a propósito.

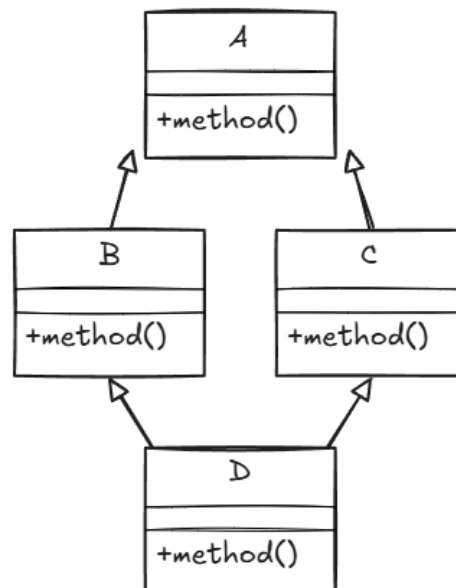


Figura 1: el problema del diamante.

D hereda de B y de C, y las dos heredan de A. Si B y C sobrescriben cada una a su manera el método `method()` de A, ¿cuál de las dos versiones hereda D? No hay respuesta buena.



Java resuelve la ambigüedad prohibiendo el caso, y ofrece otro mecanismo para cuando una clase necesita cumplir varios roles a la vez: las interfaces, que son el tema de la clase que viene.

Bibliografía sugerida para profundizar

Deitel, P., & Deitel, H. (2017). Java How to Program, Early Objects.

- Capítulo 9: Object-Oriented Programming: Inheritance. Es el capítulo completo del tema, y arranca igual que esta clase: mostrando dos clases con la mayor parte del código duplicado. La sección 9.3 trata `protected` y la 9.5, los constructores en las subclases.

Aguilar, L. J. (2011). Programación en Java. McGraw-Hill.

- Capítulo 7, sección 7.1.3 "Herencia" (p. 174) y 7.8 "Herencia: clases derivadas" (pp. 184-185), para una primera lectura.
- Capítulo 13 "Herencia" (p. 321 en adelante) para el tratamiento completo: 13.1.1 declaración de una clase derivada, 13.2 herencia pública y `protected`, 13.3 constructores en herencia y `super` (pp. 331-334), 13.6 herencia múltiple (pp. 338-339).

Azpitarte, R. et al. (2009). Introducción a la Programación Orientada a Objetos con Java.

- El capítulo dedicado a la herencia: relación de generalización y especialización, `extends`, `super` y redefinición de métodos.